

Message Passing Interface (MPI)

High Performance Computing
Master in Applied Artificial Intelligence
Nuno Lopes

Motivation

- ▶ Parallel architectures can scale by adding new nodes to the system.
 - ▶ Each node has a processor and memory independent of the other nodes.
- ▶ In the message-switching paradigm:
 - ▶ processes only send and receive messages to and from each other,
 - ▶ there is no memory sharing.
- ▶ Message Passing Interface standard, defines an API for processes to exchange data among themselves.
- ▶ Single Program Multiple Data (SPMD) approach.

Open MPI Library

- ▶ Open MPI (<https://www.open-mpi.org>) is an open-source implementation for the pc (Windows, Mac, Linux).
- ▶ Header file include:
 - ▶ `#include <mpi.h>`
- ▶ Script to compile MPI programs, wrapper for gcc:
 - ▶ `$ mpicc`
- ▶ Script to execute the program:
 - ▶ `$ mpirun (ou mpiexec)`

MPI API Initialization

- ▶ `int MPI_Init(int *argc, char ***argv)`
 - ▶ Function that must be invoked before any other MPI in the program.
 - ▶ Receives the address of the main function parameters, or NULL.
- ▶ `int MPI_Finalize()`
 - ▶ Function that terminates the MPI library in the process, from which point you should no longer invoke MPI functions.

MPI API Initialization

- ▶ `int MPI_Comm_rank(MPI_Comm comm, int *rank)`
 - ▶ Function that returns a process identifier, within the process set
 - ▶ rank is outbound parameter
- ▶ `int MPI_Comm_size(MPI_Comm comm, int *size)`
 - ▶ Function that returns the size of the process set,
 - ▶ size parameter is output.
- ▶ `MPI_COMM_WORLD`
 - ▶ Constant representing the set of all processes in an execution.

MPI Basic Example

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(NULL, NULL);
    int rank;
    int world;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &world);
    printf("Hello: rank %d, world: %d\n", rank, world);
    MPI_Finalize();
}
```

MPI Compilation and Execution

- ▶ Compile using the wrapper around the system compiler, typically gcc:

```
$ mpicc exemplo.c -o exemplo
```

- ▶ To execute with 4 process:

```
$ mpirun -n 4 ./exemplo
```

Option if necessary: `--use-hwthread-cpus`

Point-to-Point Communication

Communication between Processes

- ▶ Communication happens by sending and receiving messages.
- ▶ As the code is the same in both processes, the operations must be aligned in their execution.
 - ▶ Each process executes a different part of the same code, through "if"s.
 - ▶ Each process is identified by its rank value.
 - ▶ A process executes function to send: "Send";
 - ▶ Another process executes function to receive: "Recv".

MPI Send

```
int MPI_Send(const void *buf, int count, MPI_Datatype  
datatype, int dest, int tag, MPI_Comm comm)
```

- ▶ Buffer: memory pointer to data
- ▶ Count: number of elements in the message
- ▶ Datatype: type of data sent (MPI constant)
- ▶ Destination: rank of the destination process
- ▶ Tag: tag (integer value) used to distinguish message channels
- ▶ Comm: process group (general: MPI_COMM_WORLD)

MPI Receive

```
int MPI_Recv(void *buf, int count, MPI_Datatype  
datatype, int source, int tag, MPI_Comm comm,  
MPI_Status *status)
```

- ▶ Buffer: memory pointer to receive message
- ▶ Count: maximum number of possible elements to receive
- ▶ Datatype: type of data of the message
- ▶ Source: rank of the sender process (general: MPI_ANY_SOURCE)
- ▶ Tag: message tag (general: MPI_ANY_TAG)
- ▶ Comm: set of processes in communication (general: MPI_COMM_WORLD)
- ▶ Status: status of the result of the operation, to be consulted later.

MPI DataTypes

Table 3.1 Some Predefined MPI Datatypes

MPI datatype	C datatype
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_LONG_LONG	signed long long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	
MPI_PACKED	

MPI Count

```
int MPI_Get_count(const MPI_Status *status,  
MPI_Datatype datatype, int *count)
```

- ▶ Returns the number of elements received by the last message.
- ▶ Status: status of the receive operation
- ▶ Datatype: type of data of the message
- ▶ Count: number of elements received.

Message Exchange Example

```
const int MAX_STRING = 100;

int main(void) {
    char    greeting[MAX_STRING];
    int     comm_sz; /* Number of processes */
    int     my_rank; /* My process rank    */

    MPI_Init(NULL, NULL);
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    if (my_rank != 0) {
        sprintf(greeting, "Greetings from process %d of %d!",
                my_rank, comm_sz);
        MPI_Send(greeting, strlen(greeting)+1, MPI_CHAR, 0, 0,
                 MPI_COMM_WORLD);
    } else {
        printf("Greetings from process %d of %d!\n", my_rank,
              comm_sz);
        for (int q = 1; q < comm_sz; q++) {
            MPI_Recv(greeting, MAX_STRING, MPI_CHAR, q,
                    0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            printf("%s\n", greeting);
        }
    }

    MPI_Finalize();
    return 0;
} /* main */
```

Synchronization Model

- ▶ Send function can have a synchronous or deferred sending behaviour:
 - ▶ synchronous, if it blocks until the receiver receives the message;
 - ▶ delayed, if it returns at the sender, before the receiver receives the message (it does not block).
- ▶ The Recv function is always synchronous, that is, it blocks until it receives "the" message.
 - ▶ If the next message received does not match the reception parameters, the program may block!
 - ▶ The parameters correspond to the sender rank, the tag, the message data type and count and the comm group.

Synchronization Model

- ▶ The Ssend function has the same behaviour as Send function, but it is always synchronous and blocking, it only ends when the message reaches the destination.
 - ▶ It has the same parameters as the Send function, except for the name: MPI_Ssend

Sender/Receiver Symmetry

- ▶ For a message communication to be successful there needs to be an alignment between the sending function and receiving function.
- ▶ Both functions must be symmetrical in sender and receiver;
- ▶ If one function is not matched, the process may be blocked!
- ▶ The message must be of the same type.
- ▶ The receiver must have room to be able to receive the sent message.

Collectives

Motivation for Collectives

- ▶ Sometimes the need arises to exchange messages between all processes and not just two of them.
- ▶ This group communication can be optimised by the implementation of the library and the communication hardware itself through specific functions.
- ▶ Barrier: synchronisation of processes;
- ▶ Broadcast: sending messages to everyone, including oneself;
- ▶ Scatter/Gather: sending from one to all, sending from all to one
- ▶ Reduce: global collection to a single

MPI Barrier

```
int MPI_Barrier (MPI_Comm comm)
```

- ▶ Barrier: message for synchronisation of processes
- ▶ All processes invoke this function;
- ▶ processes are blocked until all processes in the comm group call the function.

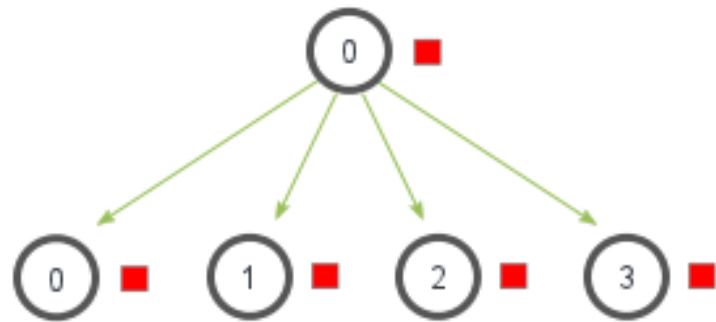
MPI Broadcast (Bcast)

```
int MPI_Bcast(void *buffer, int count, MPI_Datatype  
datatype, int root, MPI_Comm comm)
```

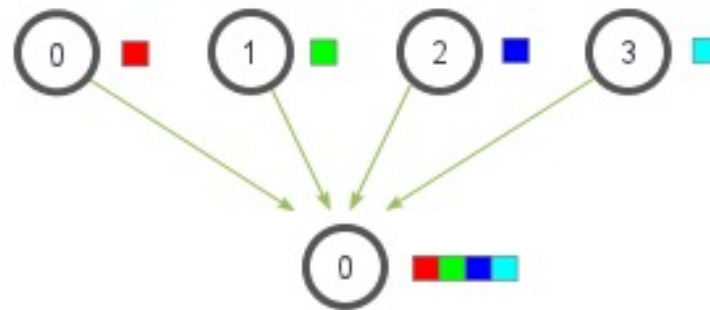
- ▶ All processes invoke this function;
 - ▶ The root process sends data;
 - ▶ The other processes receive the data.
- ▶ buffer: memory address with data
- ▶ count: number of data to send
- ▶ datatype: type of data to send
- ▶ root: rank of the process that sends the data
- ▶ comm: communication group.

Broadcast/Scatter/Gather Comparison

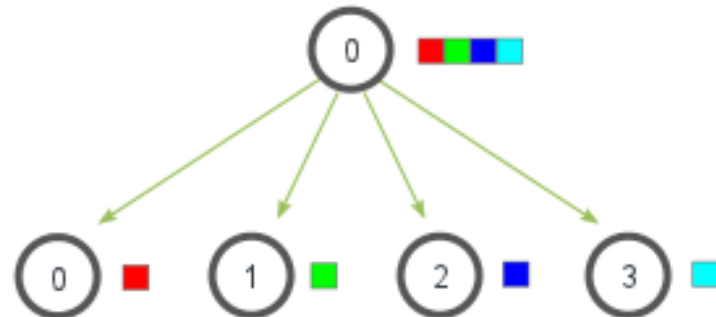
MPI_Bcast



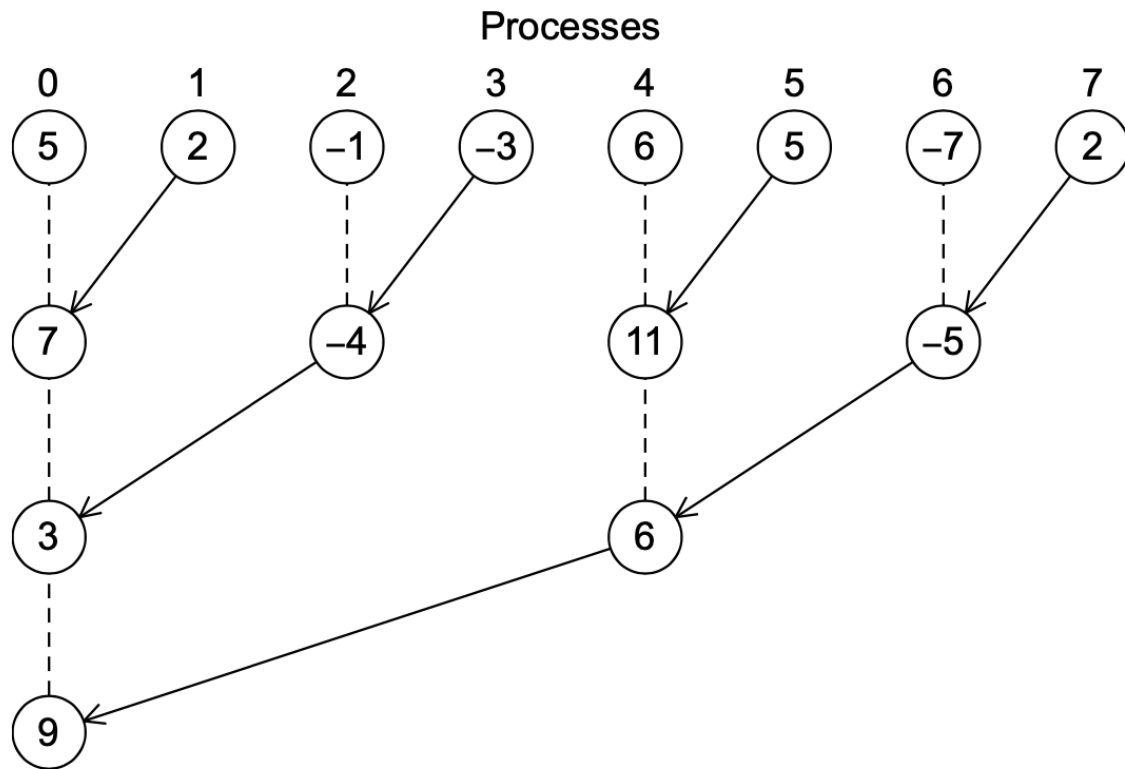
MPI_Gather



MPI_Scatter



MPI Reduce



MPI Reduce

```
int MPI_Reduce(const void *sendbuf, void *recvbuf, int count,  
MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm)
```

- ▶ Reduce function: collects a value from all processes; applies an aggregation function; merges the result in the root process.
- ▶ sendbuf: memory pointer to data to be collected in all processes;
- ▶ recvbuf: memory pointer to the final aggregate value (in the root process);
- ▶ count: number of items in the buffer;
- ▶ datatype: type of data to send;
- ▶ op: operation to apply to aggregate the results (ex: MPI_SUM)
- ▶ root: rank of the process which will have the only global result;
- ▶ comm: communication group.
- ▶ Example: `MPI_Reduce(&x, &result, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD)`

MPI Reduce - Operations

Table 3.2 Predefined Reduction Operators in MPI

Operation Value	Meaning
MPI_MAX	Maximum
MPI_MIN	Minimum
MPI_SUM	Sum
MPI_PROD	Product
MPI_LAND	Logical and
MPI_BAND	Bitwise and
MPI_LOR	Logical or
MPI_BOR	Bitwise or
MPI_LXOR	Logical exclusive or
MPI_BXOR	Bitwise exclusive or
MPI_MAXLOC	Maximum and location of maximum
MPI_MINLOC	Minimum and location of minimum